

CACHI ACTIVA

Documentación Completa de QA

Manual Testing • API Testing • Bug Reports • Métricas

Información del Proyecto	Detalles
Proyecto	Cachi Activa — Gestión de Reclamos Municipales
URL del Sistema	https://cachi-reclamos-front.vercel.app/
Tecnologías	Node.js, Express, MongoDB, JWT, Cloudinary
Tipo de Proyecto	Portfolio Personal — Full Stack
Versión del Documento	v1.0
Fecha	17/3/2026
Autor/QA Engineer	Portfolio Junior — Trainee QA

1. TEST PLAN

1.1 Objetivo del Testing

Verificar que las funcionalidades principales de Cachi Activa operen de forma correcta, segura y confiable, garantizando que los usuarios puedan registrarse, autenticarse y gestionar reclamos municipales sin errores críticos.

1.2 Alcance del Testing

Incluido en el alcance:

- Módulo de Autenticación: registro e inicio de sesión
- Módulo de Reclamos: creación, visualización y cambio de estado
- Panel de Administración: gestión de usuarios y reclamos
- Validaciones de formularios (frontend y backend)
- Seguridad básica: autenticación JWT y control de acceso por roles
- Integración con servicios externos: Cloudinary (imágenes) y Resend (emails)
- Testing de API REST: todos los endpoints documentados

Fuera del alcance:

- Testing de carga / performance (Load Testing)
- Testing de accesibilidad WCAG
- Testing automatizado (E2E con Cypress/Playwright)
- Testing en dispositivos nativos móviles

1.3 Tipos de Testing Aplicados

Tipo de Testing	Descripción	Herramienta
Testing Funcional	Verificar que cada funcionalidad opera según los requisitos	Manual / Navegador
Testing de API	Validar cada endpoint REST (request/response)	Postman / Thunder Client
Testing Negativo	Probar con datos inválidos o escenarios de error	Manual
Testing de Seguridad	Verificar autenticación, autorización y protección básica	Postman + Manual

Tipo de Testing	Descripción	Herramienta
Testing de Integración	Verificar la comunicación entre frontend, backend y servicios externos	Manual + Logs
Testing de Regresión	Verificar que bugs corregidos no vuelvan a aparecer	Manual (re-ejecución)

1.4 Criterios de Entrada (Entry Criteria)

El testing puede comenzar cuando se cumplan las siguientes condiciones:

- El entorno de staging o producción está disponible y accesible
- Los endpoints del backend están documentados y funcionando
- Existe al menos un usuario de prueba con rol 'admin' disponible
- La base de datos está poblada con datos de prueba mínimos
- Las credenciales de acceso y tokens de prueba están disponibles para el equipo de QA

1.5 Criterios de Salida (Exit Criteria)

El testing puede darse por finalizado cuando:

- El 100% de los test cases planificados han sido ejecutados
- No existen bugs críticos o de alta severidad sin resolver
- El porcentaje de test cases aprobados es mayor al 85%
- Todos los bugs reportados tienen un estado definido (abierto/resuelto/postergado)
- El Test Execution Report ha sido completado y revisado

1.6 Entorno de Testing

Componente	Detalle
Frontend (Producción)	https://cachi-reclamos-front.vercel.app/
Backend (API)	Deploy en Railway/Render — API REST con Express
Base de Datos	MongoDB Atlas (Nube)
Almacenamiento de Imágenes	Cloudinary
Servicio de Email	Resend (Transactional Email)
Navegadores Probados	Chrome 120+, Firefox 121+
Herramienta de API Testing	Postman v10 / Thunder Client (VS Code)

2. TEST CASES

Los siguientes casos de prueba cubren los flujos principales y alternativos de la aplicación, incluyendo escenarios positivos, negativos y de seguridad.

Leyenda de tipos:  Positivo |  Negativo |  Seguridad

TC-001  [Positivo] Registro exitoso con datos válidos				
Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Registro	Ninguna	1. Ir a /register 2. Ingresar nombre, email único y contraseña >= 6 chars 3. Click en Registrar	Usuario creado. Token JWT retornado. Redirige al home.	 PASS

TC-002  [Negativo] Registro con email duplicado				
Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Registro	Email ya registrado en BD	1. Ir a /register 2. Ingresar email ya existente 3. Click en Registrar	Error 400. Mensaje: 'El email ya está registrado'	 PASS

TC-003  [Negativo] Registro con contraseña menor a 6 chars				
Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Registro	Ninguna	1. Ir a /register 2. Ingresar contraseña '123' 3. Click en Registrar	Error de validación. Mensaje: contraseña mínimo 6 caracteres	 FAIL

TC-004  [Negativo] Registro con campos vacíos				
Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Registro	Ninguna	1. Ir a /register 2. No completar ningún campo 3. Click en Registrar	Error de validación en todos los campos requeridos	 PASS

TC-005 [Positivo] Login exitoso con credenciales válidas

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Login	Usuario registrado	1. Ir a /login 2. Ingresar email y contraseña correctos 3. Click en Ingresar	Token JWT generado. Usuario redirigido al dashboard.	 PASS

TC-006 [Negativo] Login con contraseña incorrecta

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Login	Usuario registrado	1. Ir a /login 2. Ingresar email correcto y contraseña errónea 3. Click en Ingresar	Error 401. Mensaje: 'Contraseña incorrecta'	 PASS

TC-007 [Negativo] Login con email no registrado

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Login	Ninguna	1. Ir a /login 2. Ingresar email inexistente 3. Click en Ingresar	Error 401. Mensaje: 'Usuario no encontrado'	 PASS

TC-008 [Positivo] Crear reclamo con todos los campos completos

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Reclamos	Usuario logueado con token válido	1. Ir a /crear-reclamo 2. Completar título, descripción, categoría 3. Subir imagen opcional 4. Click en Enviar	Reclamo creado con estado 'Pendiente'. Email enviado al admin y al usuario.	 PASS

TC-009 [Seguridad] Crear reclamo sin autenticación

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
--------	----------------	-------	--------------------	--------

Reclamos	Usuario no logueado	1. Intentar POST /api/reportes sin token 2. Enviar datos del reclamo	Error 401. Acceso denegado.	 PASS
----------	---------------------	--	-----------------------------	---

TC-010 [Negativo] Crear reclamo sin título

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Reclamos	Usuario logueado	1. Completar solo descripción y categoría 2. Dejar título vacío 3. Enviar formulario	Error de validación: el título es obligatorio	 FAIL

TC-011 [Positivo] Visualizar listado de reclamos

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Reclamos	Existen reclamos en la BD	1. Ir a la sección de reclamos 2. Verificar que se muestra el listado	Se muestran todos los reclamos ordenados por fecha descendente	 PASS

TC-012 [Positivo] Cambiar estado de un reclamo

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Admin	Usuario con rol 'admin' logueado	1. Ingresar al panel admin 2. Seleccionar un reclamo 3. Cambiar estado a 'En Proceso'	Estado actualizado correctamente en la BD.	 PASS

TC-013 [Seguridad] Acceder al panel admin sin ser admin

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Admin	Usuario con rol 'user' logueado	1. Intentar acceder a rutas /admin con token de usuario normal	Error 403. Acceso denegado por rol insuficiente.	 FAIL

TC-014 [Negativo] Subir imagen en formato no válido

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Reclamos	Usuario logueado	1. Crear reclamo 2. Adjuntar archivo .pdf o .exe 3. Enviar formulario	Error de validación: solo se permiten imágenes (jpg, png, webp)	✗ FAIL

TC-015 [Seguridad] Usar token expirado

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Seguridad	Token JWT vencido (>24h)	1. Realizar petición autenticada con token expirado 2. Enviar request a /api/reportes	Error 403. Mensaje: 'Token inválido o expirado'	✓ PASS

TC-016 [Seguridad] Inyección en campo de texto

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Seguridad	Usuario logueado	1. En el campo título ingresar: <code><script>alert('XSS')</script></code> 2. Enviar reclamo	El script no se ejecuta. El texto se guarda como string plano.	✓ PASS

TC-017 [Positivo] Eliminar reclamo existente

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Admin	Reclamo existente, admin logueado	1. Panel admin 2. Seleccionar reclamo 3. Click en Eliminar 4. Confirmar	Reclamo eliminado de la BD. Ya no aparece en el listado.	✓ PASS

TC-018 [Positivo] Eliminar usuario existente

Módulo	Precondiciones	Pasos	Resultado Esperado	Estado
Admin	Usuario existente, admin logueado	1. Panel admin > Usuarios 2. Seleccionar usuario 3. Eliminar	Usuario eliminado correctamente.	✓ PASS

3. TEST EXECUTION REPORT

Reporte de ejecución simulada de los test cases definidos en la sección anterior. La ejecución fue realizada manualmente sobre el entorno de producción.

 **Datos de Ejecución**

Fecha de ejecución: 17/3/2026

Ejecutado por: QA Engineer (Trainee)

Entorno: Producción — <https://cachi-reclamos-front.vercel.app/>

Navegador: Google Chrome 120.0

Ciclo de testing: Ciclo 1 — Testing inicial

ID	Módulo	Título (resumido)	Resultado Esperado	Resultado Actual	Estado
TC-001	Registro	Registro exitoso con datos válidos	Usuario creado. Token JWT retornado. Redirige al home.	Comportamiento correcto ✓	 PASS
TC-002	Registro	Registro con email duplicado	Error 400. Mensaje: 'El email ya está registrado'	Comportamiento correcto ✓	 PASS
TC-003	Registro	Registro con contraseña menor a 6 chars	Error de validación. Mensaje: contraseña mínimo 6 caracteres	No coincide con lo esperado X	 FAIL
TC-004	Registro	Registro con campos vacíos	Error de validación en todos los campos requeridos	Comportamiento correcto ✓	 PASS
TC-005	Login	Login exitoso con credenciales válidas	Token JWT generado. Usuario redirigido al dashboard.	Comportamiento correcto ✓	 PASS
TC-006	Login	Login con contraseña incorrecta	Error 401. Mensaje: 'Contraseña incorrecta'	Comportamiento correcto ✓	 PASS
TC-007	Login	Login con email no registrado	Error 401. Mensaje: 'Usuario no encontrado'	Comportamiento correcto ✓	 PASS
TC-008	Reclamos	Crear reclamo con todos los campos compl...	Reclamo creado con estado 'Pendiente'. Email enviado al admi...	Comportamiento correcto ✓	 PASS
TC-009	Reclamos	Crear reclamo sin autenticación	Error 401. Acceso denegado.	Comportamiento correcto ✓	 PASS
TC-010	Reclamos	Crear reclamo sin título	Error de validación: el título es obligatorio	No coincide con lo esperado X	 FAIL

ID	Módulo	Título (resumido)	Resultado Esperado	Resultado Actual	Estado
TC-011	Reclamos	Visualizar listado de reclamos	Se muestran todos los reclamos ordenados por fecha descendentes...	Comportamiento correcto ✓	✓ PASS
TC-012	Admin	Cambiar estado de un reclamo	Estado actualizado correctamente en la BD.	Comportamiento correcto ✓	✓ PASS
TC-013	Admin	Acceder al panel admin sin ser admin	Error 403. Acceso denegado por rol insuficiente.	No coincide con lo esperado ✗	✗ FAIL
TC-014	Reclamos	Subir imagen en formato no válido	Error de validación: solo se permiten imágenes (jpg, png, we...	No coincide con lo esperado ✗	✗ FAIL
TC-015	Seguridad	Usar token expirado	Error 403. Mensaje: 'Token inválido o expirado'	Comportamiento correcto ✓	✓ PASS
TC-016	Seguridad	Inyección en campo de texto	El script no se ejecuta. El texto se guarda como string plan...	Comportamiento correcto ✓	✓ PASS
TC-017	Admin	Eliminar reclamo existente	Reclamo eliminado de la BD. Ya no aparece en el listado.	Comportamiento correcto ✓	✓ PASS
TC-018	Admin	Eliminar usuario existente	Usuario eliminado correctamente.	Comportamiento correcto ✓	✓ PASS

4. BUG REPORTS

Los siguientes defectos fueron identificados durante la ejecución del ciclo de testing. Cada bug incluye información suficiente para que el equipo de desarrollo pueda reproducirlo y corregirlo.

ID	Título	Módulo	Severidad	Estado	Tipo
BUG-001	Contraseña de 3 chars no es rechazada en el frontend	Registro	Media	Abierto	Defecto Funcional
BUG-002	Campo título de reclamo no muestra error si está vacío	Reclamos	Alta	Abierto	Defecto Funcional
BUG-003	No hay restricción de rol en rutas del panel admin	Seguridad / Admin	Crítica	Abierto	Defecto Funcional
BUG-004	Formato de archivo no validado al subir imagen	Reclamos	Media	Abierto	Defecto Funcional
BUG-005	Email de confirmación no indica número de seguimiento usable	Notificaciones	Baja	Abierto	Defecto Funcional
BUG-006	Error 500 sin mensaje descriptivo cuando falla la conexión a MongoDB	General	Alta	Abierto	Defecto Funcional

BUG-001 — Contraseña de 3 chars no es rechazada en el frontend | Severidad: Media | Estado: Abierto

Módulo: Registro

Pasos para reproducir:
1. Ir a /register 2. Completar nombre y email 3. Ingresar contraseña '123' 4. Click en Registrar

Resultado esperado: Mensaje de error: 'La contraseña debe tener al menos 6 caracteres'

Resultado actual: El formulario envía la petición al backend sin validar. El backend la rechaza, pero el mensaje de error no se muestra en la UI.

Notas / Recomendación: *Falta validación en el cliente (frontend). Impacta la UX.*

BUG-002 — Campo título de reclamo no muestra error si está vacío | Severidad: Alta | Estado: Abierto

Módulo: Reclamos

Pasos para reproducir:

1. Loguearse 2. Ir a crear reclamo 3. Dejar título vacío 4. Completar descripción y categoría 5. Enviar

Resultado esperado: Mensaje de validación: 'El título es obligatorio'

Resultado actual: El formulario no muestra error visible. El backend retorna 500 sin mensaje claro para el usuario.

Notas / Recomendación: *El modelo tiene required: true pero el frontend no valida antes de enviar.*

BUG-003 — No hay restricción de rol en rutas del panel admin | Severidad: Crítica | Estado: Abierto

Módulo: Seguridad / Admin

Pasos para reproducir:

1. Loguearse con usuario normal (rol: user) 2. Obtener token JWT 3. Hacer PATCH /api/users/:id/rol con ese token

Resultado esperado: Error 403: acceso denegado (solo admin puede cambiar roles)

Resultado actual: La petición es procesada exitosamente. Un usuario normal puede cambiar roles de otros usuarios.

Notas / Recomendación: *El middleware verifyToken valida autenticación pero no verifica el rol. VULNERABILIDAD DE SEGURIDAD.*

BUG-004 — Formato de archivo no validado al subir imagen | Severidad: Media | Estado: Abierto

Módulo: Reclamos

Pasos para reproducir:

1. Loguearse 2. Crear reclamo 3. Adjuntar un archivo .pdf 4. Enviar

Resultado esperado: Error: 'Solo se permiten imágenes (jpg, png, webp)'

Resultado actual: El archivo .pdf se sube a Cloudinary sin validación. Cloudinary lo rechaza y la app no maneja el error correctamente.

Notas / Recomendación: *Agregar filtro de mimetype en la config de Multer/Cloudinary.*

BUG-005 — Email de confirmación no indica número de seguimiento usable | Severidad: Baja | Estado: Abierto

Módulo: Notificaciones

Pasos para reproducir:

1. Crear un reclamo exitosamente 2. Revisar el email de confirmación recibido

Resultado esperado: El número de reclamo debe ser un identificador amigable o un link directo

Resultado actual: El email muestra el ObjectId de MongoDB (ej: 6612f3a4c9...) como 'Número de Reclamo', que no es útil para el usuario.

Notas / Recomendación: *Considerar usar un número correlativo o un código más amigable.*

BUG-006 — Error 500 sin mensaje descriptivo cuando falla la conexión a MongoDB | Severidad: Alta | Estado: Abierto

Módulo: General

Pasos para reproducir:

1. Simular caída de MongoDB (detener el servicio) 2. Intentar hacer login

Resultado esperado: Mensaje de error claro: 'Servicio no disponible, intente más tarde'

Resultado actual: La app retorna un error 500 genérico sin mensaje al usuario. En logs aparece el error de Mongoose.

Notas / Recomendación: *Agregar manejo de errores global (middleware de error) en Express.*

5. MÉTRICAS DE CALIDAD

Métrica	Valor
Total de Test Cases ejecutados	18
Test Cases PASS (Aprobados)	13
Test Cases FAIL (Fallidos)	5
Porcentaje de éxito (Pass Rate)	72.2%
Bugs encontrados (total)	6
Bugs Críticos	1 (BUG-003 — Seguridad de roles)
Bugs de Alta severidad	2 (BUG-002, BUG-006)
Bugs de Media severidad	2 (BUG-001, BUG-004)
Bugs de Baja severidad	1 (BUG-005)
Test Cases de Seguridad	3
Módulos con mayor cantidad de bugs	Seguridad/Autenticación, Validaciones UI
Ciclos de testing realizados	1 (Ciclo inicial)
Test Cases de API	16
API Tests PASS	14 / 16 (87.5%)

Análisis de Resultados

El Pass Rate inicial del 72.2% refleja un sistema funcional en sus flujos principales (login, registro, creación de reclamos).

El BUG-003 (ausencia de verificación de rol en rutas admin) es el hallazgo más crítico y debe resolverse antes de cualquier release.

Los 4 FAIL restantes son principalmente validaciones de UI faltantes, que afectan la experiencia pero no comprometen la seguridad del dato.

Se recomienda realizar un segundo ciclo de testing luego de las correcciones para validar la regresión.

6. TESTING DE API

6.1 Configuración del Entorno (Postman)

Para ejecutar los tests de API se recomienda configurar las siguientes variables de entorno en Postman:

Variable	Valor de ejemplo
{{base_url}}	https://[tu-backend].railway.app/api
{{token_user}}	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (token de usuario normal)
{{token_admin}}	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... (token de admin)
{{user_id}}	64abc123... (ID de MongoDB del usuario de prueba)
{{reporte_id}}	64def456... (ID de MongoDB del reclamo de prueba)

6.2 Uso del Token JWT

🔑 Cómo autenticarse en las peticiones

1. Realizar POST /api/users/login con credenciales válidas
2. Copiar el valor del campo 'token' de la respuesta
3. En cada petición protegida, agregar el header:

Key: Authorization

Value: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Nota: El token expira en 24 horas. Si obtienes error 403, obtén un nuevo token.

6.3 Casos de Prueba de API

Endpoint	Descripción	Body / Auth	Resp. Esperada	Estado
POST /api/users/register	Registro exitoso	{"nombre": "Ana Test", "email": "ana@test.com", "password": "secure123"}	201 + token	✓
POST /api/users/register	Email duplicado	{"nombre": "Ana", "email": "ana@test.com", "password": "secure123"}	400 El email ya está registrado	✓

Endpoint	Descripción	Body / Auth	Resp. Esperada	Estado
POST /api/users/register	Password corta	{"nombre": "Ana", "email": "ana2@test.com", "password": "123"}	400 contraseña mínimo 6 chars	✗
POST /api/users/login	Login exitoso	{"email": "ana@test.com", "password": "secure123"}	200 + user + token	✓
POST /api/users/login	Contraseña incorrecta	{"email": "ana@test.com", "password": "wrong"}	401 Contraseña incorrecta	✓
POST /api/users/login	Email inexistente	{"email": "noexiste@test.com", "password": "abc123"}	401 Usuario no encontrado	✓
GET /api/users/	Listar usuarios (con token admin)	— (Bearer token admin)	200 Array de usuarios sin password	✓
GET /api/users/	Sin token de autenticación	— (Sin header Authorization)	401 No estás autorizado	✓
POST /api/reportes	Crear reclamo autenticado	Form-data: titulo, descripcion, categoria + token	201 + reclamo creado	✓
POST /api/reportes	Sin token	Form-data: titulo, descripcion, categoria	401 No estás autorizado	✓
POST /api/reportes	Token inválido	Authorization: Bearer TOKENINVALIDO	403 Token inválido o expirado	✓
GET /api/reportes	Listar todos los reclamos	— (Público)	200 Array de reclamos	✓
PATCH /api/reportes/:id	Cambiar estado (admin)	{"estado": "En Proceso"} + token admin	200 reporte actualizado	✓
PATCH /api/reportes/:id	ID inexistente	{"estado": "Resuelto"} + token admin	404 Reporte no encontrado	✓
DELETE /api/reportes/:id	Eliminar reclamo (admin)	— + token admin	200 Reporte eliminado	✓

Endpoint	Descripción	Body / Auth	Resp. Esperada	Estado
			correctamente	
PATCH /api/users/:id /rol	Cambiar rol (usuario normal)	{"rol": "admin"} + token USER	403 Acceso denegado	×

6.4 Casos de Error HTTP más comunes

Código HTTP	Nombre	Cuándo ocurre	Mensaje esperado
400	Bad Request	Datos inválidos, campos faltantes o formato incorrecto	{ message: 'El email ya está registrado' }
401	Unauthorized	Token ausente o credenciales incorrectas	{ message: 'No estás autorizado (Falta Token)' }
403	Forbidden	Token inválido, expirado o rol insuficiente	{ message: 'Token inválido o expirado' }
404	Not Found	El recurso (reporte, usuario) no existe en la BD	{ message: 'Reporte no encontrado' }
500	Internal Server Error	Error inesperado del servidor (bug en el código)	{ message: error.message }

7. 📁 Estructura de Carpetas en GitHub

Se recomienda organizar la documentación de QA en el repositorio de la siguiente manera:

Ruta / Archivo	Descripción
qa/	Carpeta raíz de toda la documentación de QA
qa/README.md	Índice general del proyecto de QA con resumen ejecutivo
qa/test-plan/TEST_PLAN.md	Plan de testing completo (objetivo, alcance, criterios)
qa/test-cases/TC_AUTH.md	Test cases del módulo de autenticación
qa/test-cases/TC_RECLAMOS.md	Test cases del módulo de reclamos
qa/test-cases/TC_ADMIN.md	Test cases del panel de administración
qa/test-cases/TC_SEGURIDAD.md	Test cases de seguridad y autorización
qa/execution-reports/REPORT_CICLO1.md	Reporte de ejecución del primer ciclo de testing
qa/bug-reports/BUGS.md	Listado de bugs con severidad, pasos y estado
qa/api-testing/POSTMAN_COLLECTION.json	Colección exportada de Postman con todos los requests
qa/api-testing/ENVIRONMENT.json	Variables de entorno de Postman (sin credenciales reales)
qa/metrics/METRICAS.md	Dashboard de métricas: pass rate, bugs, cobertura
qa/docs/QA_FULL_DOCUMENTATION.docx	Este documento Word completo

💡 Consejo para Portfolio

Agrega un badge en el README.md principal del proyecto con el resultado del testing:

![Tests](https://img.shields.io/badge/Tests-13%2F18%20PASS-green)

![Bugs](https://img.shields.io/badge/Bugs-6%20encontrados-red)

Esto hace que la documentación de QA sea inmediatamente visible para los reclutadores.

8. Lecciones Aprendidas

8.1 Aspectos positivos del proyecto

- La separación en capas (routes / controllers / services) facilita enormemente identificar en qué capa se encuentra un bug.
- El uso de JWT para autenticación es una práctica correcta. La implementación del middleware verifyToken es limpia y reutilizable.
- La integración con Cloudinary para el manejo de imágenes simplifica mucho la lógica del backend y centraliza la transformación de imágenes.
- El envío de emails automáticos al crear reclamos mejora la experiencia del usuario y puede construirse sobre esa base para agregar notificaciones de estado.
- El uso de toJSON para limpiar automáticamente _id, __v y password es una buena práctica de seguridad y limpieza de respuestas.

8.2 Áreas de mejora identificadas

- Validaciones en el Frontend: muchas validaciones de formulario solo existen en el backend. La experiencia del usuario mejora significativamente si se valida también en el cliente antes de enviar la petición.
- Control de acceso por roles (RBAC): el middleware verifyToken solo verifica que el token existe y es válido, pero no verifica el rol del usuario. Esto dejó expuesto el endpoint de cambio de roles a cualquier usuario autenticado.
- Manejo de errores global: la ausencia de un middleware global de errores en Express hace que algunos errores retornen respuestas 500 con mensajes de Mongoose directamente, lo cual puede exponer información sensible.
- Mensajes de error en la UI: cuando el backend retorna un error, el frontend no siempre lo muestra de forma clara al usuario. Mejorar el feedback visual de errores es prioritario.

8.3 Lo que aprendí haciendo este testing

- La importancia de testear no solo el 'happy path' (todo va bien) sino también los casos negativos y de borde (qué pasa cuando algo falla).
- Un bug de seguridad (como BUG-003) puede estar presente en un sistema que 'funciona perfectamente' desde la perspectiva del usuario normal. El testing de seguridad es imprescindible.
- Documentar mientras se testea es más eficiente que intentar recordar lo que se hizo después.
- La comunicación entre QA y desarrollo mejora cuando los bug reports son claros, tienen pasos reproducibles y distinguen claramente el resultado esperado del actual.

9. Posibles Mejoras del Sistema

9.1 Mejoras de Seguridad (Prioridad Alta)

- Implementar middleware de verificación de rol: crear `verifyAdmin` que compruebe `req.user.rol === 'admin'` antes de ejecutar operaciones administrativas.
- Rate limiting: agregar `express-rate-limit` para prevenir ataques de fuerza bruta en los endpoints de login y registro.
- Sanitización de inputs: usar `express-validator` o similar para validar y sanitizar todos los campos que llegan del cliente.
- Helmet.js: agregar el paquete `Helmet` para configurar headers HTTP de seguridad automáticamente.

9.2 Mejoras de Funcionalidad

- Paginación en el listado de reclamos: cuando la BD crezca, devolver todos los reclamos de una vez se volverá lento. Implementar paginación (`page`, `limit`).
- Filtros y búsqueda: permitir filtrar reclamos por categoría, estado y fecha desde el frontend.
- Actualización de estado con notificación al usuario: cuando el admin cambia el estado del reclamo, enviar un email al vecino que lo creó.
- Perfil de usuario editable: permitir al usuario cambiar su nombre y contraseña.
- Número de reclamo amigable: reemplazar el `ObjectId` de MongoDB por un número correlativo (ej: `#0001`, `#0002`) en las comunicaciones con el usuario.

9.3 Mejoras de Calidad y Mantenibilidad

- Testing automatizado: implementar tests unitarios con `Jest` para las funciones de los servicios (`userService`, `reporteService`, `emailService`).
- Testing E2E: implementar tests de extremo a extremo con `Cypress` o `Playwright` para los flujos principales.
- CI/CD Pipeline: integrar los tests automáticos en un pipeline de `GitHub Actions` que se ejecute en cada push.
- Variables de entorno documentadas: crear un archivo `.env.example` con todas las variables requeridas (sin valores reales) para facilitar la configuración del proyecto.
- Logging estructurado: reemplazar los `console.log` por una librería de logging (como `Winston` o `Pino`) que permita distinguir niveles (`info`, `warn`, `error`) y centralizar los logs.

9.4 Mejoras de UX/UI

- Feedback visual en tiempo real: mostrar mensajes de error/éxito directamente en el formulario, sin depender de alertas del navegador.

- Indicador de estado de la subida de imagen: mostrar una barra de progreso mientras se sube la imagen a Cloudinary.
- Validación de formulario en el cliente: implementar validaciones inmediatas (mientras el usuario tipea) para mejorar la experiencia.

Documentación generada para Portfolio de QA

Cachi Activa — Sistema de Gestión de Reclamos Municipales

Total: 18 Test Cases | 6 Bug Reports | 16 API Tests | Ciclo 1 Completado